CSDF Mini Project

Topic : Design and develop a tool for digital forensic of images.

Code :

```
"""
Image Forensics Tool (CSDF Mini Project)
----------------------------------------
Usage: python image_forensics.py <image_file>

What it does:
1. File information (name, size, timestamps)
2. Cryptographic hashes (MD5, SHA-1, SHA-256) for evidence integrity
3. File signature (magic number) vs extension check
4. Hidden / appended data check (data after JPEG end marker)
5. EXIF metadata extraction (camera, date, software, GPS)
6. Error Level Analysis (ELA) to highlight possible tampering
7. Writes a text report + ELA image

Requires: pip install pillow
"""
import sys
import os
import io
import hashlib
import datetime

from PIL import Image, ImageChops, ImageEnhance
from PIL.ExifTags import TAGS, GPSTAGS

SIGNATURES = {
b"\xff\xd8\xff": "JPEG",
b"\x89PNG\r\n\x1a\n": "PNG",
b"GIF87a": "GIF",
b"GIF89a": "GIF",
b"BM": "BMP",
}
EXT_MAP = {".jpg": "JPEG", ".jpeg": "JPEG", ".png": "PNG", ".gif": "GIF", ".bmp": "BMP"}



def file_info(path):
st = os.stat(path)
fmt = "%Y-%m-%d %H:%M:%S"
return {
"File name": os.path.basename(path),
"Size (bytes)": st.st_size,
"Last modified": datetime.datetime.fromtimestamp(st.st_mtime).strftime(fmt),
"Last accessed": datetime.datetime.fromtimestamp(st.st_atime).strftime(fmt),
}
```

```
def hashes(path):
md5, sha1, sha256 = hashlib.md5(), hashlib.sha1(), hashlib.sha256()
with open(path, "rb") as f:
for chunk in iter(lambda: f.read(8192), b""):
md5.update(chunk)
sha1.update(chunk)
sha256.update(chunk)
return {"MD5": md5.hexdigest(), "SHA-1": sha1.hexdigest(), "SHA-256": sha256.hexdigest()}



def signature_check(path):
with open(path, "rb") as f:
head = f.read(16)
real = "Unknown"
for sig, name in SIGNATURES.items():
if head.startswith(sig):
real = name
break
ext = os.path.splitext(path)[1].lower()
claimed = EXT_MAP.get(ext, "Unknown")
status = "MATCH" if real == claimed else "MISMATCH (possible renamed/disguised file)"
return {"Extension says": claimed, "Actual signature": real, "Result": status}



def hidden_data_check(path):
with open(path, "rb") as f:
data = f.read()
if data.startswith(b"\xff\xd8"):
end = data.rfind(b"\xff\xd9")
extra = len(data) - (end + 2)
if end == -1:
return {"Result": "JPEG end marker not found (file may be corrupted)"}
if extra > 0:
return {"Result": f"SUSPICIOUS: {extra} bytes found after JPEG end marker"}
return {"Result": "No data appended after JPEG end marker"}
if data.startswith(b"\x89PNG"):
end = data.rfind(b"IEND")
extra = len(data) - (end + 8)
if extra > 0:
return {"Result": f"SUSPICIOUS: {extra} bytes found after PNG IEND chunk"}
return {"Result": "No data appended after PNG IEND chunk"}
return {"Result": "Check not applicable for this format"}



def _gps_to_deg(value, ref):
d, m, s = [float(x) for x in value]
deg = d + m / 60 + s / 3600
return -deg if ref in ("S", "W") else deg



def exif_data(img):
out = {}
try:
exif = img._getexif()
```

```
except Exception:
exif = None
if not exif:
return {"EXIF": "No EXIF metadata found (stripped, screenshot, or edited/re-saved image)"}
wanted = ["Make", "Model", "Software", "DateTime", "DateTimeOriginal",
"ExposureTime", "FNumber", "ISOSpeedRatings", "Orientation"]
for tag_id, val in exif.items():
tag = TAGS.get(tag_id, tag_id)
if tag in wanted:
out[tag] = str(val)
if tag == "GPSInfo":
gps = {GPSTAGS.get(k, k): v for k, v in val.items()}
try:
lat = _gps_to_deg(gps["GPSLatitude"], gps["GPSLatitudeRef"])
lon = _gps_to_deg(gps["GPSLongitude"], gps["GPSLongitudeRef"])
out["GPS"] = f"{lat:.6f}, {lon:.6f}"
except Exception:
out["GPS"] = "Present but could not be decoded"
if "Software" in out:
out["Note"] = "Software tag present - image may have been edited/processed"
return out or {"EXIF": "EXIF present but no common tags found"}



def ela(img, out_path, quality=90):
rgb = img.convert("RGB")
buf = io.BytesIO()
rgb.save(buf, "JPEG", quality=quality)
buf.seek(0)
resaved = Image.open(buf)
diff = ImageChops.difference(rgb, resaved)
extrema = diff.getextrema()
max_diff = max(e[1] for e in extrema)
scale = 255.0 / max_diff if max_diff else 1
diff = ImageEnhance.Brightness(diff).enhance(scale)
diff.save(out_path)
# simple statistic: mean brightness of ELA image
gray = diff.convert("L")
pixels = list(gray.getdata())
mean = sum(pixels) / len(pixels)
bright = sum(1 for p in pixels if p > 200) / len(pixels) * 100
if bright > 5:
verdict = "HIGH - large regions with very different compression levels; manual inspection advised"
elif bright > 1:
verdict = "MEDIUM - some regions differ; inspect the ELA image"
else:
verdict = "LOW - uniform error levels; no obvious tampering"
return {"Max error difference": max_diff, "Mean ELA brightness": round(mean, 2),
"Pixels with high error (%)": round(bright, 2), "Tampering suspicion": verdict}



def section(title, d):
lines = [f"\n=== {title} ==="]
for k, v in d.items():
lines.append(f"{k:28}: {v}")
return "\n".join(lines)
```

```
def main():
if len(sys.argv) != 2 or not os.path.isfile(sys.argv[1]):
print("Usage: python image_forensics.py <image_file>")
sys.exit(1)
path = sys.argv[1]
base = os.path.splitext(os.path.basename(path))[0]
ela_path = f"ela_{base}.png"
report_path = f"forensic_report_{base}.txt"


img = Image.open(path)
img_props = {"Format": img.format, "Mode": img.mode, "Dimensions": f"{img.width} x {img.height}"}


parts = [
"DIGITAL IMAGE FORENSIC REPORT",
f"Generated on: {datetime.datetime.now():%Y-%m-%d %H:%M:%S}",
section("1. FILE INFORMATION", file_info(path)),
section("2. IMAGE PROPERTIES", img_props),
section("3. HASH VALUES (integrity)", hashes(path)),
section("4. FILE SIGNATURE CHECK", signature_check(path)),
section("5. HIDDEN DATA CHECK", hidden_data_check(path)),
section("6. EXIF METADATA", exif_data(img)),
section("7. ERROR LEVEL ANALYSIS", ela(img, ela_path)),
f"\nELA image saved as: {ela_path}",
]
report = "\n".join(parts)
print(report)
with open(report_path, "w", encoding="utf-8") as f:
f.write(report)
print(f"\nReport saved as: {report_path}")



if __name__ == "__main__":
main()
```

OUTPUT 1:



DIGITAL IMAGE FORENSIC REPORT

Generated on: 2026-10-22 08:56:49

=== 1. FILE INFORMATION ===

File name : photo.jpg

Size (bytes) : 864440

Last modified : 2026-10-22 08:55:15

Last accessed : 2026-10-22 08:56:02

=== 2. IMAGE PROPERTIES ===

Format : JPEG

Mode : RGB

Dimensions : 3024 x 4032

=== 3. HASH VALUES (integrity) ===

MD5 : c60a93f655623485276945958eaf3099

SHA-1 : 775a22417c71671cc2cace9167befc7f78bf8793

SHA-256 : 37111cc7ef2cdbe0ef60d91e066c75377bb54b7612136119855c1624bc8a74ab

=== 4. FILE SIGNATURE CHECK ===

Extension says : JPEG

Actual signature : JPEG

Result : MATCH

=== 5. HIDDEN DATA CHECK ===

Result : No data appended after JPEG end marker

=== 6. EXIF METADATA ===

EXIF : No EXIF metadata found (stripped, screenshot, or edited/re-saved image)

=== 7. ERROR LEVEL ANALYSIS ===

Max error difference : 6

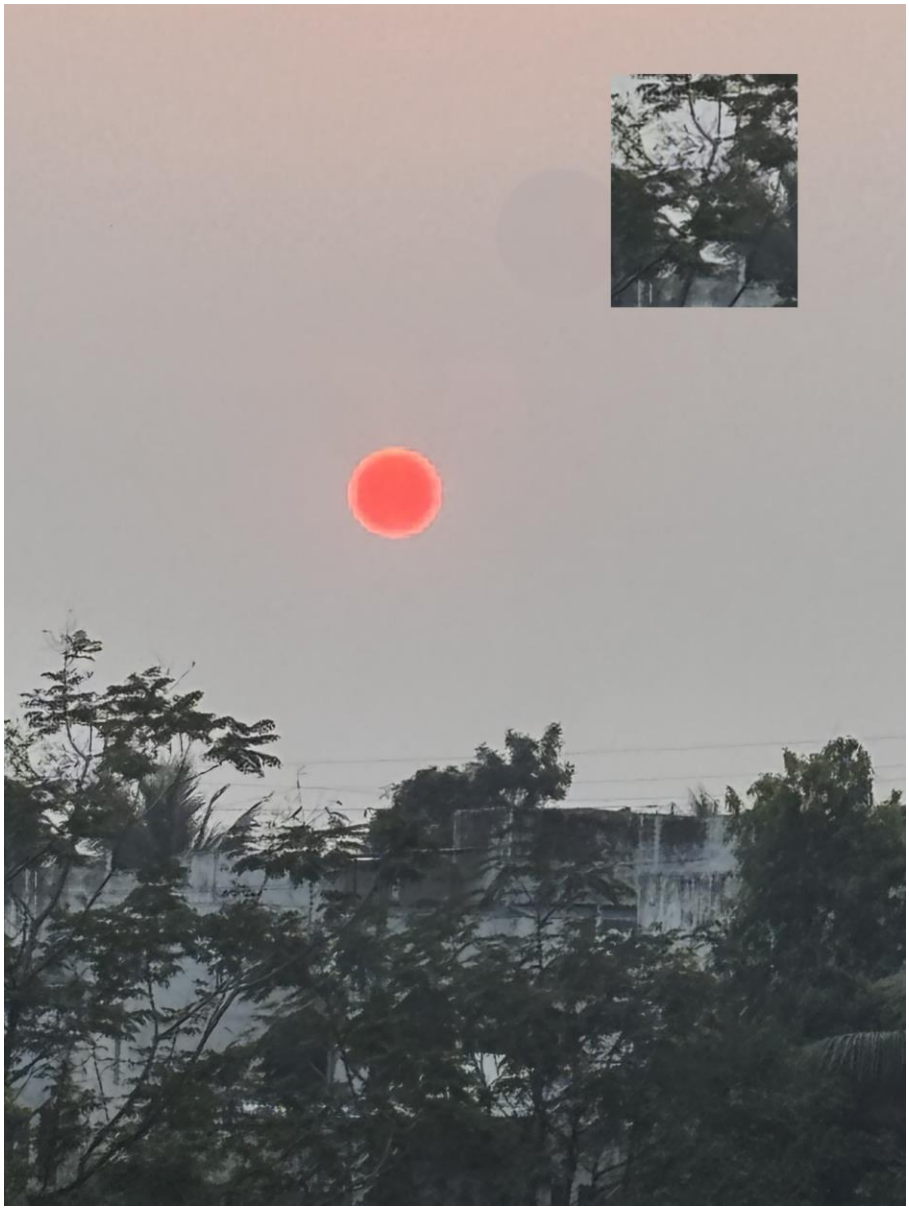
Mean ELA brightness : 6.29

Pixels with high error (%) : 0.0

Tampering suspicion : LOW - uniform error levels; no obvious tampering

ELA image saved as: ela_photo.png

Output 2 :



DIGITAL IMAGE FORENSIC REPORT

Generated on: 2026-10-22 09:22:40

=== 1. FILE INFORMATION ===

File name : tampered_photo.jpg

Size (bytes) : 662388

Last modified : 2026-10-22 09:21:25

Last accessed : 2026-10-22 09:22:19

=== 2. IMAGE PROPERTIES ===

Format : JPEG

Mode : RGB

Dimensions : 1932 x 2576

=== 3. HASH VALUES (integrity) ===

MD5 : 7134ba3818e64d89c94e35ae9b56ee1d

SHA-1 : b4ea6061bc35dd7f0ca41884deb26b8b9c6fbb46

SHA-256 : d50a2dcffb54501c37c50a6f505e7be42945caaf67ba063640a6c154044b5e22

=== 4. FILE SIGNATURE CHECK ===

Extension says : JPEG

Actual signature : JPEG

Result : MATCH

=== 5. HIDDEN DATA CHECK ===

Result : No data appended after JPEG end marker

=== 6. EXIF METADATA ===

Software : Adobe Photoshop 25.0

DateTime : 2026:10:22 09:30:00

Note : Software tag present - image may have been edited/processed

=== 7. ERROR LEVEL ANALYSIS ===

Max error difference : 12

Average block error : 0.069

Suspicious blocks : 119 of 4800 (2.48%)

Tampering suspicion : MEDIUM - localised regions differ (possible paste/clone); inspect the ELA image

ELA image saved as: ela_tampered_photo.png